Data-Driven Clothing for Interactive Applications: a PCA + Neural-Network Reimplementation

UE24CS352A Machine Learning - Mini-Project | Team: [Name 1 (SRN)], [Name 2 (SRN)] | Problem no.: [problem no.] | Code: [GitHub URL]

1. Problem statement

Physics-based cloth simulation gives realistic clothing but is too slow for interactive use; the reference paper (Xue & Wu, Stanford CS 229) reports roughly 180 s per simulated coat on a CPU. We learn a fast surrogate: given the 14 joint-rotation quaternions of a body pose (56 inputs), predict the offsets of the 2,170 coat vertices from a linearly skinned coat (2,170 x 3 = 6,510 outputs). We compare a linear-regression baseline, a neural network that predicts the offsets directly, and a neural network that predicts PCA coefficients of the offsets (k = 256, 128, 64).

2. Dataset

The paper's data (Blender skinning and PhysBAM simulation) is not publicly available, so we generated a **synthetic stand-in with identical shapes**. 10,000 random poses are drawn as unit quaternions. Each target is the sum of a linear term (mimicking skinning), a smooth nonlinear tanh term (standing in for physics effects skinning cannot capture) and small Gaussian noise; the generator is seeded and fully reproducible. As in the paper we remove suspect examples: the 2% with the largest mean squared offset distance (threshold 0.091, Fig. 1a) were dropped, leaving 9,800 examples split 6,000 train / 2,000 validation / 1,800 test. Absolute errors are therefore *not comparable with the paper's*; comparisons between models are.

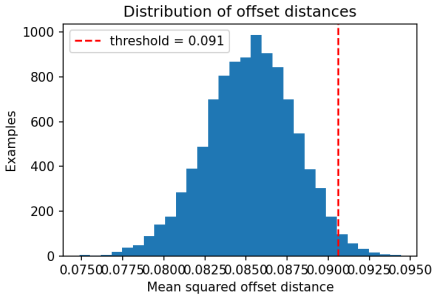


Fig. 1a: distribution of offset distances and the cleaning threshold.

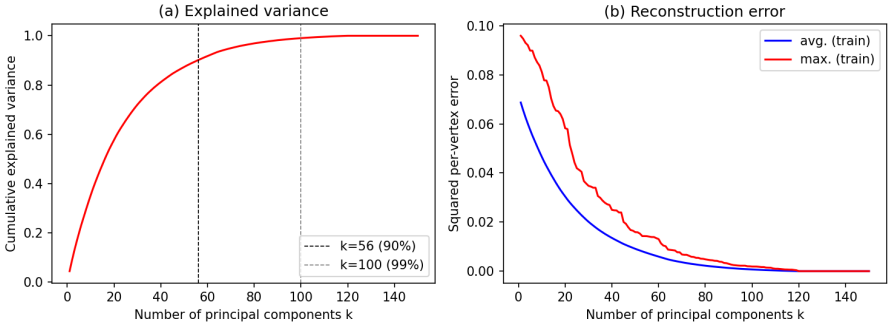


Fig. 1b: PCA explained variance (left) and reconstruction error vs. number of components k (right).

3. Approach

PCA. The 6,510 outputs are strongly correlated, so we fit PCA (via SVD) on the *training* offsets only and reconstruct a mesh as x = sum$_i$ c$_i$ u$_i$ + mean. 56 components explain 90% of the variance and 100 explain 99% (Fig. 1b). The reconstruction floor (mean squared per-vertex error on training data) is 4.8e-3 at k = 64 but only 2.9e-6 at k = 128: 64 components are too few, 128 are ample.

Models. (0) Linear regression: a network with no hidden layers (56 to 6,510). (1) An MLP with ReLU layers 128-256 predicting the 6,510 offsets directly. (2-4) The same MLP predicting k PCA coefficients, followed by a *frozen* "PC layer" whose weights are the principal components and whose bias is the mean; only the MLP is trained.

Training and selection. MSE loss on the reconstructed offsets, Adam, batch size 32, 1,000 epochs, learning rate 1e-3 for all models, seed 0. The best epoch and best model are chosen on validation MSE; the test set is used only for reporting.

4. Implementation overview

PyTorch. `cloth_pipeline.py` holds the data generator, PCA, models and training loop; `prepare_data.py`, `pca_analysis.py`, `train_models.py` and `evaluate.py` run the steps in order (data, PCA, training of five models, test-set evaluation with per-vertex errors and timing). One Colab notebook (`cloth_mini_project.ipynb`) reproduces the whole pipeline. Code and README are in a private GitHub repository.

5. Results

Model	Test MSE	Sq. per-vertex err.	Relative err. (%)	Trainable params	Test MSE at lr 1e-4
0 Linear regression	6.62e-3	1.98e-2	23.32	371,070	6.49e-3
1 MLP to offsets *	**1.38e-4**	**4.13e-4**	**0.48**	**1,713,390**	**5.03e-4**
2 MLP to 256 PCA coeffs	1.45e-4	4.35e-4	0.51	106,112	2.04e-3
3 MLP to 128 PCA coeffs	1.35e-4	4.04e-4	0.48	73,216	2.44e-3
4 MLP to 64 PCA coeffs	1.77e-3	5.31e-3	6.24	56,768	2.90e-3

Table 1: test-set results (1,800 examples; lr 1e-3, 1,000 epochs, all models). * best on validation MSE. The last column is an earlier run of the same models at lr 1e-4 (see below). Single run, seed 0.

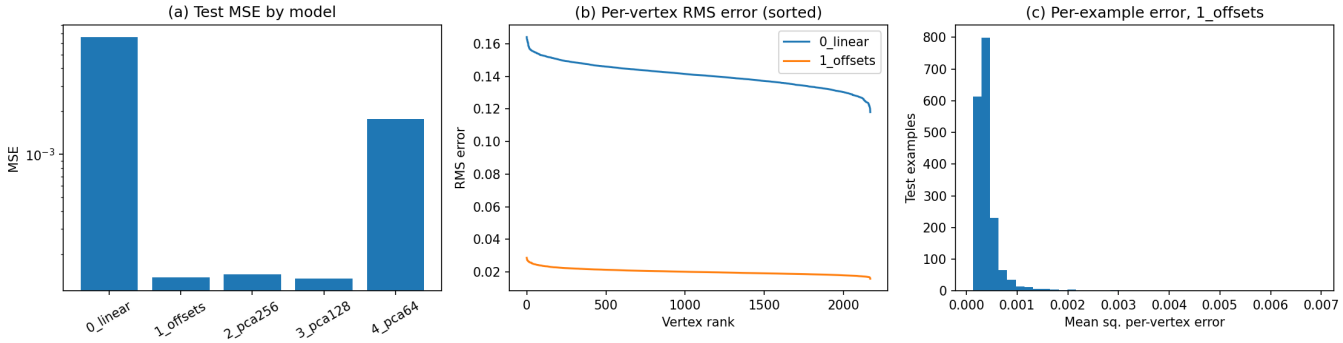
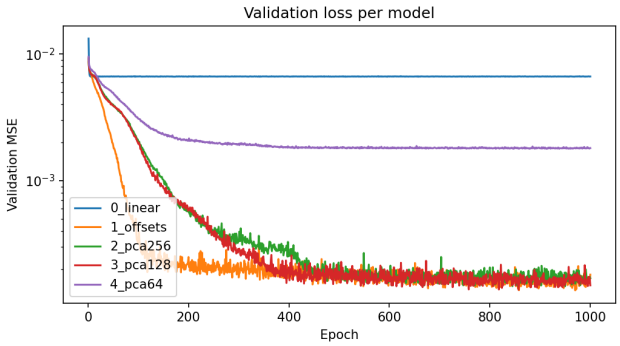


Fig. 2: (a) test MSE per model (log scale); (b) per-vertex RMS error, sorted, for the linear baseline and the direct MLP; (c) per-example error of the direct MLP.



Observations

- **Networks beat linear.** Relative error drops from 23.3% to 0.48% (98% lower test MSE); error is spread evenly over vertices (Fig. 2b).
- **Direct, PCA-128 and PCA-256 are comparable** (1.35e-4 to 1.45e-4; single seed, differences of a few percent, so no ranking is claimed). PCA-128 needs 73k trainable parameters versus 1.7M.
- **PCA-64 is limited by reconstruction**, as its floor (4.8e-3) predicts.
- **Learning rate mattered.** At 1e-4 the PCA models underfit (train error close to test error) even after 1,000 epochs; 1e-3 for all models removed most of the gap (Table 1, Fig. 3). Standardising the PCA coefficients did not help.
- **Speed.** Inference takes about 0.1 ms per mesh on a CPU (timings vary between runs). We did not run a cloth simulator, so no speed-up is claimed beyond the paper's own figure.

Fig. 3: validation loss per model (lr 1e-3).

6. Conclusions

A PCA-based neural network predicts the synthetic coat offsets with about 0.5% relative error, far below a linear baseline (23%), at roughly 0.1 ms per mesh. 128 PCA coefficients suffice and 64 do not. Unlike the paper, we found no clear advantage of PCA over direct prediction; plausible reasons, which we did not test, are our synthetic data and the single random seed. **Limitations:** synthetic data, one seed per model, no temporal coherence, no 3D rendering of meshes. **Future work:** real simulated data, several seeds, and the convolutional pixel-based approach mentioned in the paper. Reference: K. Xue and J. Wu, *Data-Driven Clothing for Interactive Applications*, Stanford CS 229.